Project Pt. 2

EVALUATING DATA STRUCTURES USING A HEALTHCARE DATASET

Presented By

Timonique Crighton | Arianna Clayton | Tracy Amuzu | Rene Ngobi

AGENDA

1. Problem Overview
2. Patient Record Model
3. System Architecture
4. Dynamic Array
5. Linked List
6. Hash Map
7. Queue
8. Benchmark Methodology
9. Benchmark Results
10. Performance Analysis
11. Conclusion
12. The End

Problem Overview

Healthcare systems store large amounts of patient data

Need efficient ways to:

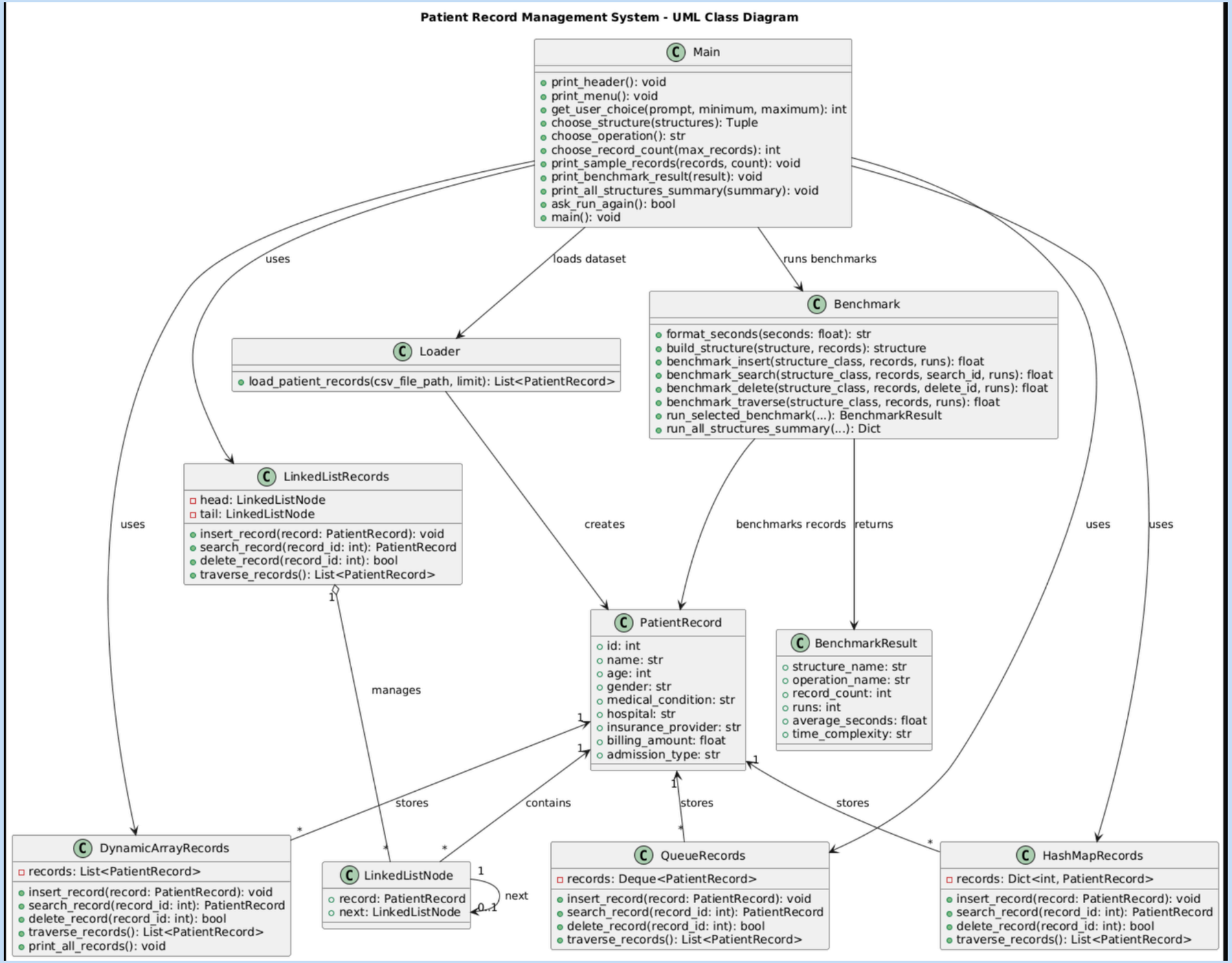
- Insert records
- Search by ID
- Delete records
- Traverse data

Different data structures impact performance

Patient Record Model

- ID (unique identifier)
- name
- age
- gender
- medical_condition
- hospital
- insurance_provider
- billing_amount
- admission_type

SYSTEM ARCHITECTURE



Dynamic Array

Efficient data storage with fast appending and simple traversal for managing collections of data.

Dynamic Array Design	**self.records = [] # dynamic array** • Stores elements in order of insertion • Supports fast access using index • Flexible and easy to use

Limitations

- No direct fast search → must scan (O(n))
- Inserting in middle requires shifting (O(n))
- Less efficient for frequent modifications

Core Operations

Insert (Append) *self.records.append(value)*	• Adds element to end of array • Fast most of the time • Runtime: O(1) amortized
Traverse (Visit All Elements) *for item in self.records:* *print(item)*	• Adds element to end of array • Fast most of the time • Runtime: O(1) amortized
Search (Find Element) *for item in self.records:* *if item == target:* *return True*	• Checks each element • No shortcuts • Runtime: O(n)
Print (All) *print(self.records)*	• Outputs entire array • Runtime: O(n)
Python list	• Uses built-in list structure • Automatically resizes as elements are added

linked List

Efficient structure for storing patient records with flexible memory and fast insertions.

linked list	self.head = None # start of list • Node-based structure • Each node → patient record + pointer • Dynamically grows (no fixed size) • No index-based access

Limitations

- No direct fast search → must scan (O(n))
- Slower searching (no indexing)
- Extra memory for pointers
- Deletion requires traversal

Core Operations

Insert (Add records) *new_node.next = self.head* *self.head = new_node*	• Adds record to list • Very fast insertion • Runtime: O(1)
Traverse (Visit All records) *current = self.head* *while current:* *print(current.data)* *current = current.next*	• Goes through each node • Runtime: O(n)
Search (Find record) *current = self.head* *while current:* *if current.data == target:* *return True* *current = current.next*	• Checks each node • No shortcuts (no indexing) • Runtime: O(n)
delete (remove record) *current = self.head* *prev = None*	• Must find record first • Updates pointers to remove node • Runtime: O(n)

HASH- BASED STRUCTURE

Hash Map Design

self.records = {} # hash map

- Uses a Python dictionary
- Key → record.id
- Value → full PatientRecord
- Best for fast ID-based search
- Stores records as key-value pairs

Limitations

- Not ideal when order matters
- Uses extra memory for keys
- Worst case can become slower if many hash collisions happen
- Best when records need fast lookup by ID

Core Operations	
Insert Reords self.records[record.id] = record	• Adds or updates record by ID • Runtime: O(1) average
Search Records return self.records.get(record_id)	• Finds patient directly using ID • No scanning needed • Runtime: O(1) average
Delete Record if record_id in self.records: del self.records[record_id]	• Checks if ID exists • Removes key-value pair • Runtime: O(1) average
Traverse Records return list(self.records.values())	• Returns all patient records • Must visit every stored record • Runtime: O(n)

QUEUE

Purpose: Appointment/Admission scheduling ensuring a First-Come, First-Served processing!

Queue (Deque)	from collections import deque • ideal for real-time hospital flow • Fast enqueue & dequeue O(1)
Queue Design	self.records:Deque[AdmissionQueueEntry] = deque() self.admission_counter = 0 • Stores patients in arrival order • admission_counter ensures strict FIFO tracking

Core Operations

Enqeue(Insert Patient)

self.records.append(AdmissionQueueEntry(...))

- Adds patient to end of queue
- Assigns admission order

Dequeue (Process Patient)

self.records.popleft()

- Removes patient from front
- Next patient gets served

Peek (Next Patient)

self.records[0]

- View next patient without removing/reordering

Limitations:

-No direct search → must scan O(n)

-Delete requires rebuilding queue

-Best when order matters, not random access

BENCHMARK METHODOLOGY

Dataset sizes tested:

- 100
- 1,000
- 5,000
- 10,000

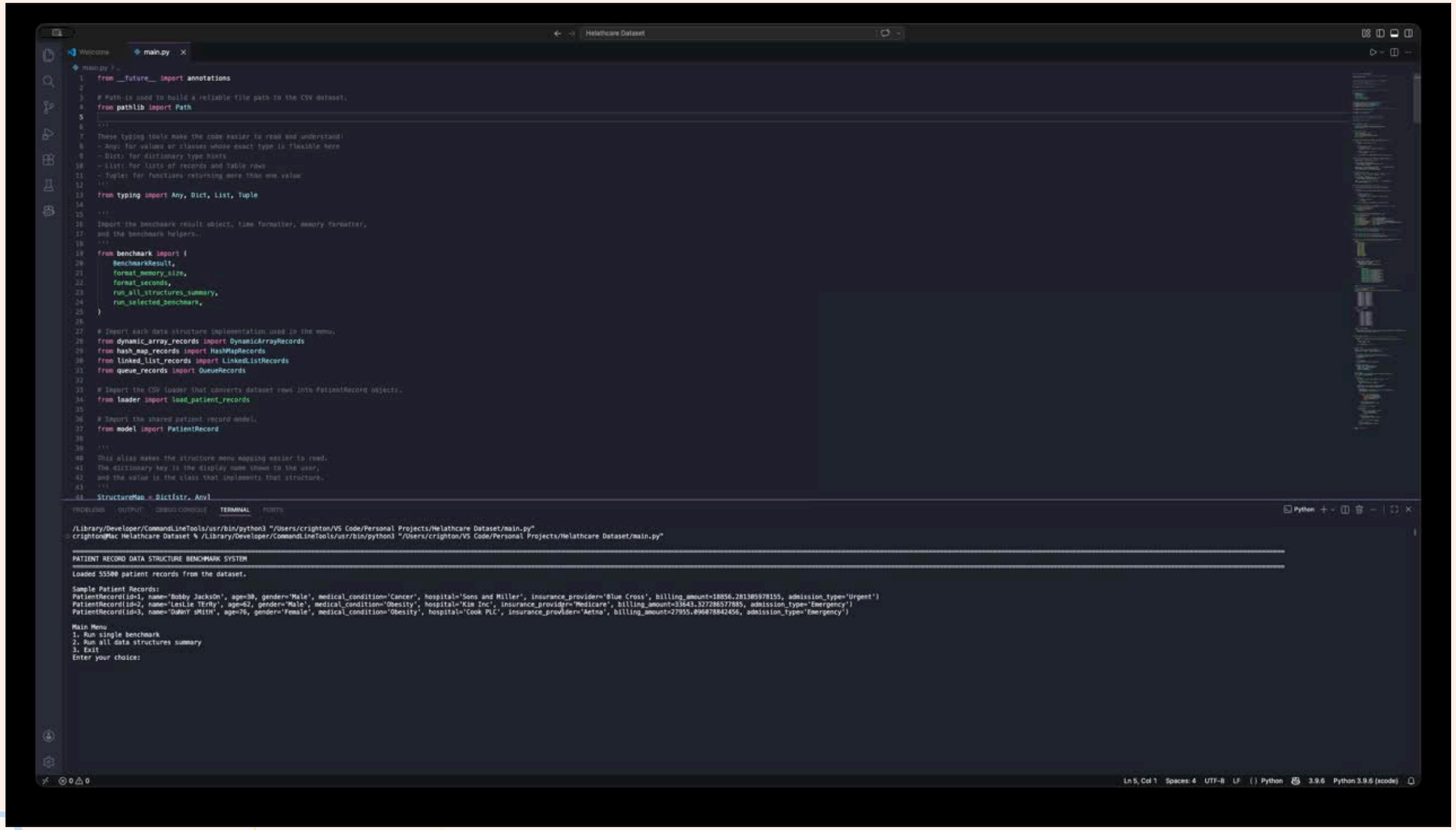
Each test run 5 times

Measured:

- Execution time
- Memory usage

Used Python time + tracemalloc

BENCHMARK RESULTS



BENCHMARK RESULTS

```
Main Menu
1. Run single benchmark
2. Run all data structures summary
3. Exit
Enter your choice: 2

Dataset size available: 1 to 55500 records
How many records should be used? 100

All Data Structures Summary
==========================================================================================================================================
Measured time, measured memory, and theoretical Big-O are shown separately.
Data Structure            Insert Time     Insert Memory   Insert Big-O    Search Time     Search Memory   Search Big-O    Delete Time     Delete Memory   Delete Big-O    Traverse Time   Traverse Memory Traverse Big-O
------------------------------------------------------------------------------------------------------------------------------------------
Dynamic Array             0.00001965 sec  1.15 KB         O(1) amortized  0.00000262 sec  219.00 B        O(n)            0.00000628 sec  284.00 B        O(n)            0.00000125 sec  937.00 B        O(n)
Linked List               0.00009820 sec  11.14 KB        O(1)            0.00000358 sec  84.00 B         O(n)            0.00000792 sec  171.00 B        O(n)            0.00000843 sec  950.00 B        O(n)
Queue / Admission Queue   0.00009518 sec  10.40 KB        O(1)            0.00000310 sec  158.00 B        O(n)            0.00001269 sec  185.00 B        O(n)            0.00000533 sec  1.21 KB         O(n)
Hash Map                  0.00001553 sec  6.81 KB         O(1) average    0.00000042 sec  84.00 B         O(1) average    0.00000049 sec  83.00 B         O(1) average    0.00000193 sec  1.03 KB         O(n)
==========================================================================================================================================
Note: Big-O is theoretical growth, not a conversion of measured time or measured memory. Results shown are averages across 5 runs.

Dataset size available: 1 to 55500 records
How many records should be used? 1000

All Data Structures Summary
==========================================================================================================================================
Measured time, measured memory, and theoretical Big-O are shown separately.
Data Structure            Insert Time     Insert Memory   Insert Big-O    Search Time     Search Memory   Search Big-O    Delete Time     Delete Memory   Delete Big-O    Traverse Time   Traverse Memory Traverse Big-O
------------------------------------------------------------------------------------------------------------------------------------------
Dynamic Array             0.00041510 sec  8.64 KB         O(1) amortized  0.00008611 sec  48.00 B         O(n)            0.00046945 sec  168.00 B        O(n)            0.00001147 sec  7.87 KB         O(n)
Linked List               0.00239813 sec  143.75 KB       O(1)            0.00006246 sec  0.00 B          O(n)            0.00008808 sec  0.00 B          O(n)            0.00010626 sec  8.59 KB         O(n)
Queue / Admission Queue   0.00156098 sec  164.99 KB       O(1)            0.00002792 sec  72.00 B         O(n)            0.00013968 sec  201.00 B        O(n)            0.00003572 sec  8.81 KB         O(n)
Hash Map                  0.00013248 sec  54.10 KB        O(1) average    0.00000047 sec  0.00 B          O(1) average    0.00000054 sec  0.00 B          O(1) average    0.00000481 sec  7.98 KB         O(n)
==========================================================================================================================================
Note: Big-O is theoretical growth, not a conversion of measured time or measured memory. Results shown are averages across 5 runs.
How many records should be used? 5000

All Data Structures Summary
==========================================================================================================================================
Measured time, measured memory, and theoretical Big-O are shown separately.
Data Structure            Insert Time     Insert Memory   Insert Big-O    Search Time     Search Memory   Search Big-O    Delete Time     Delete Memory   Delete Big-O    Traverse Time   Traverse Memory Traverse Big-O
------------------------------------------------------------------------------------------------------------------------------------------
Dynamic Array             0.00123681 sec  40.89 KB        O(1) amortized  0.00024062 sec  48.00 B         O(n)            0.00163125 sec  168.00 B        O(n)            0.00000975 sec  39.12 KB        O(n)
Linked List               0.00609883 sec  737.43 KB       O(1)            0.00014858 sec  9.00 B          O(n)            0.00027392 sec  0.00 B          O(n)            0.00027141 sec  40.86 KB        O(n)
Queue / Admission Queue   0.00525697 sec  899.16 KB       O(1)            0.00011238 sec  76.00 B         O(n)            0.00050164 sec  201.00 B        O(n)            0.00011928 sec  41.06 KB        O(n)
Hash Map                  0.00051562 sec  216.10 KB       O(1) average    0.00000048 sec  0.00 B          O(1) average    0.00000046 sec  0.00 B          O(1) average    0.00001645 sec  39.23 KB        O(n)
==========================================================================================================================================
Note: Big-O is theoretical growth, not a conversion of measured time or measured memory. Results shown are averages across 5 runs.
How many records should be used? 10000

All Data Structures Summary
==========================================================================================================================================
Measured time, measured memory, and theoretical Big-O are shown separately.
Data Structure            Insert Time     Insert Memory   Insert Big-O    Search Time     Search Memory   Search Big-O    Delete Time     Delete Memory   Delete Big-O    Traverse Time   Traverse Memory Traverse Big-O
------------------------------------------------------------------------------------------------------------------------------------------
Dynamic Array             0.00280091 sec  83.17 KB        O(1) amortized  0.00041383 sec  48.00 B         O(n)            0.00262918 sec  168.00 B        O(n)            0.00002103 sec  78.18 KB        O(n)
Linked List               0.01070132 sec  1.44 MB         O(1)            0.00028534 sec  9.00 B          O(n)            0.00053017 sec  32.00 B         O(n)            0.00050822 sec  83.14 KB        O(n)
Queue / Admission Queue   0.01059223 sec  1.78 MB         O(1)            0.00022789 sec  76.00 B         O(n)            0.00094978 sec  211.00 B        O(n)            0.00021772 sec  83.34 KB        O(n)
Hash Map                  0.00100678 sec  432.10 KB       O(1) average    0.00000043 sec  0.00 B          O(1) average    0.00000052 sec  0.00 B          O(1) average    0.00003119 sec  78.29 KB        O(n)
==========================================================================================================================================
Note: Big-O is theoretical growth, not a conversion of measured time or measured memory. Results shown are averages across 5 runs.
```

PERFORMANCE ANALYSIS

STRUCTURE	INSERT	SEARCH	DELETE	TRAVERSE
DYNAMIC ARRAY	O(1)	O(N)	O(N)	O(N)
LINKED LIST	O(1)	O(N)	O(N)	O(N)
HASH MAP	O(1)	O(1)	O(1)	O(1)
QUEUE	O(1)	O(N)	O(N)	O(N)

CONCLUSION

Lessons Learned:

- Big-O explains trends, but measured results show the real trade-off
- Memory usage matters, not just runtime
- A queue is more meaningful when tied to a real use case
- Clean interfaces make benchmarking easier

Challenges

- Turning a real dataset into a simplified record model
- Keeping the comparison fair across all structures
- Handling queue deletion for the project
- Debuggin integration issues across multiple files

THANK YOU

Any questions?